



EduGenie

Google Gemini Powered Learning Assistant

PROJECT DOCUMENTATION

FastAPI • Google Gemini API • HTML/CSS/JavaScript

Version 1.0.0

Implementation-aligned documentation prepared from the supplied project source package, the reference documentation, and the recorded live application interface.

1. Project Overview

EduGenie is a web-based learning assistant designed to help students understand academic topics, practice through automatically generated questions, condense educational text, and follow structured learning paths. The application combines a lightweight browser interface with a Python FastAPI backend and Google Gemini for generative responses.

The project follows the structure and intent of the supplied reference documentation while expanding it into a complete implementation document covering architecture, modules, API design, user workflow, validation, testing, deployment, security, and future scope.

1.1 Problem Statement

Students often need several separate activities during learning: asking a question, understanding a difficult concept, creating practice questions, reducing long notes into key points, and deciding what to study next. These activities can be time-consuming when handled manually. EduGenie brings these tasks into one simple interface and provides AI-assisted educational output through a single FastAPI application.

1.2 Proposed Solution

EduGenie provides five student-facing learning functions through a unified web interface: **Q&A, Explain, Generate Quiz, Summarize, and Recommend Learning Path**. The browser sends structured JSON requests to FastAPI endpoints. Feature modules prepare focused prompts and the shared AI client communicates with Google Gemini. The application also contains graceful fallback handling for temporary AI-service failures so that the demonstration interface remains usable.

1.3 Objectives

- Provide a simple AI-assisted educational interface for students.
- Answer academic questions in clear, student-friendly language.
- Explain difficult concepts using structured, beginner-oriented prompts.
- Generate three-question MCQ practice sets with four options and explanations.
- Summarize supplied educational text without intentionally introducing unrelated information.
- Produce a staged learning path from fundamentals to revision.
- Expose a clean API architecture that can be extended with additional learning modules.
- Keep credentials outside the source code through environment variables.

2. Key Features

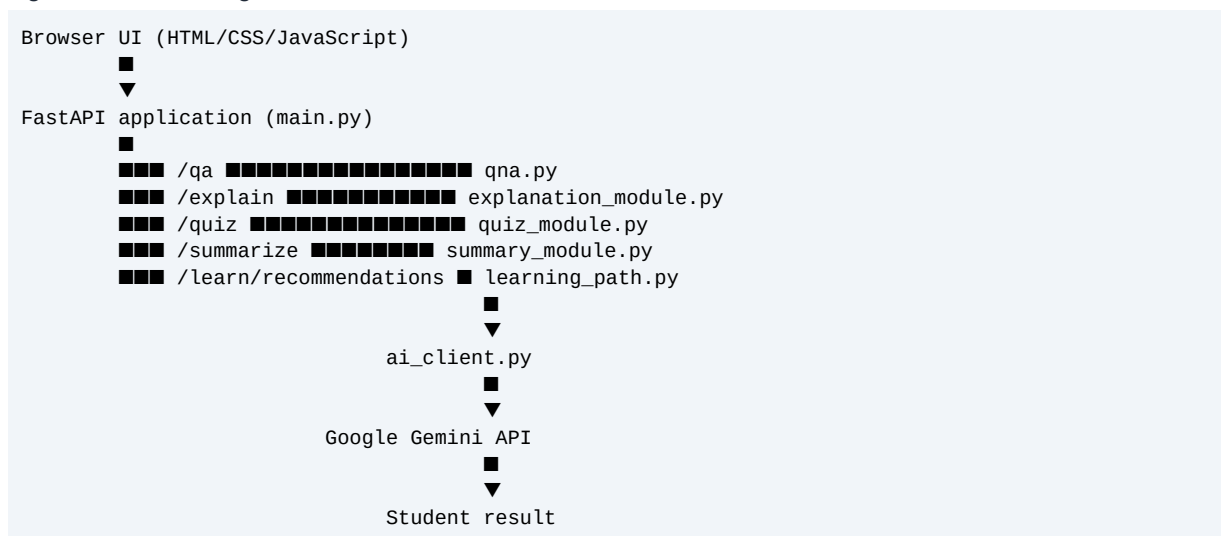
Feature	Purpose
Q&A	Students enter a question and receive a concise educational response.
Explain	Students enter a topic and receive a structured explanation with definition, main idea, steps, example-oriented
Generate Quiz	The system requests exactly three MCQs, four options per question, one correct answer, and a short explanation
Summarize	Educational text is sent for concise summarization. The implementation also contains a deterministic sentence
Recommend Learning Path	A topic is converted into a staged beginner-to-advanced learning plan with practice and final revision stages.

3. Technology Stack

Layer	Technology / Implementation
Programming language	Python
Backend framework	FastAPI
ASGI server	Uvicorn
Validation	Pydantic v2
AI integration	Google Gemini API through google-genai SDK
Frontend	HTML5, CSS3, vanilla JavaScript
Templating	Jinja2 / FastAPI Jinja2Templates
Configuration	python-dotenv and environment variables
Deployment	Render (live web deployment)
Version control	GitHub

4. System Architecture

The application uses a modular client-server architecture. The browser is responsible for collecting user input and rendering results. FastAPI provides the application and API layer. Feature modules contain task-specific prompting and validation logic. A shared AI client isolates Gemini communication and fallback behavior.



4.1 Request Flow

- The user selects a learning task and enters a question, topic, or text.
- JavaScript selects the corresponding REST endpoint and sends JSON using fetch().
- FastAPI validates the request with Pydantic models.
- The relevant feature module constructs an educational prompt.
- The shared AI client calls the configured Gemini model and optionally requests structured JSON for quizzes.
- The response is returned as JSON to the browser and rendered in the Result section.
- Temporary service failures are detected by the AI client and routed to clean fallback responses.

5. Project Structure

```
EduGenie/  
  main.py  
  config.py  
  ai_client.py  
  qna.py  
  explanation_module.py  
  quiz_module.py  
  summary_module.py  
  learning_path.py  
  requirements.txt  
  templates/  
    index.html  
  static/  
    style.css  
  tests/  
    test_api.py  
  .gitignore
```

5.1 Module Responsibilities

File	Responsibility
main.py	Creates the FastAPI app, serves the UI, defines request models and exposes REST endpoints.
config.py	Loads environment variables and application configuration such as the Gemini model and input length limit.
ai_client.py	Centralizes Gemini API calls, JSON generation, temporary-error detection, and fallback behavior.
qna.py	Builds the Q&A prompt and invokes the shared AI client.
explanation_module.py	Builds structured educational explanations; optionally supports a local transformer model when enabled.
quiz_module.py	Builds MCQ prompts, requests a JSON schema, and validates the generated quiz structure.
summary_module.py	Builds summarization prompts and delegates generation to the shared AI client.
learning_path.py	Builds personalized learning-path prompts and delegates generation.
templates/index.html	Defines the browser interface and JavaScript request logic.
static/style.css	Defines responsive visual styling, forms, result cards, quiz blocks, loading state, and errors.
tests/test_api.py	Reserved test module included in the project structure.

6. Backend API Design

Method	Endpoint	Purpose	Request
GET	/	Serves EduGenie web interface	Browser request
GET	/health	Health/status check	None
POST	/qa	Answers a student question	{"question": "..."}
POST	/explain	Explains a topic	{"topic": "..."}
POST	/quiz	Generates 3 MCQs	{"topic": "..."}
POST	/summarize	Summarizes supplied text	{"text": "..."}
POST	/learn/recommendations	Generates a learning path	{"topic": "..."}

6.1 Request Validation

FastAPI uses Pydantic models with required string fields and minimum length validation. The feature modules additionally enforce a maximum input size configured through `MAX_INPUT_CHARS`, which defaults to 12,000 characters. Empty input is rejected before AI generation.

6.2 Quiz JSON Contract

The quiz feature requests structured JSON. The schema requires a questions array containing exactly three objects. Each object requires a question, exactly four options, a `correct_answer`, and an explanation. The implementation verifies that the correct answer is one of the returned options before the result is sent to the frontend.

7. AI Integration and Prompt Engineering

The shared `ai_client.py` module uses the Google GenAI Python SDK. The API key and model name are loaded from environment variables. The application is configured to use the `GEMINI_MODEL` environment value, with **gemini-3.8-flash** as the source-code default when no model value is supplied.

7.1 Feature-specific Prompting

- Q&A; prompt: requests concise, accurate, student-friendly answers.
- Explanation prompt: requests definition, main idea, step-by-step explanation, simple example guidance, and important points.
- Quiz prompt: explicitly constrains question count, option count, uniqueness of the correct answer, and explanation fields.
- Summary prompt: asks the model to preserve important concepts, remove repetition, use simple language, and avoid introducing unrelated facts.
- Learning-path prompt: requests beginner, intermediate, advanced, timeline, practice, project/resource guidance, and final revision.

7.2 Structured Output

For quiz generation, `response_mime_type` is set to `application/json` and a response schema is supplied to the Gemini client. The returned text is parsed as JSON, with a small cleanup path for fenced JSON responses. Invalid structures are rejected rather than silently displayed.

7.3 Resilience and Fallback

The AI client detects temporary conditions such as quota exhaustion, rate limiting, HTTP 429/503-style service conditions, and temporary unavailability. In those cases, the application uses predefined demonstration-safe fallback behavior. The fallback is intentionally separate from the normal Gemini generation path and keeps technical service details out of the student-facing result for supported fallback cases.

8. Frontend Design and User Experience

The frontend is intentionally lightweight: a single responsive HTML page, one CSS file, and browser-native JavaScript. No separate frontend framework or build step is required. The interface presents the five functions through a task selector and dynamically changes the input label and placeholder.

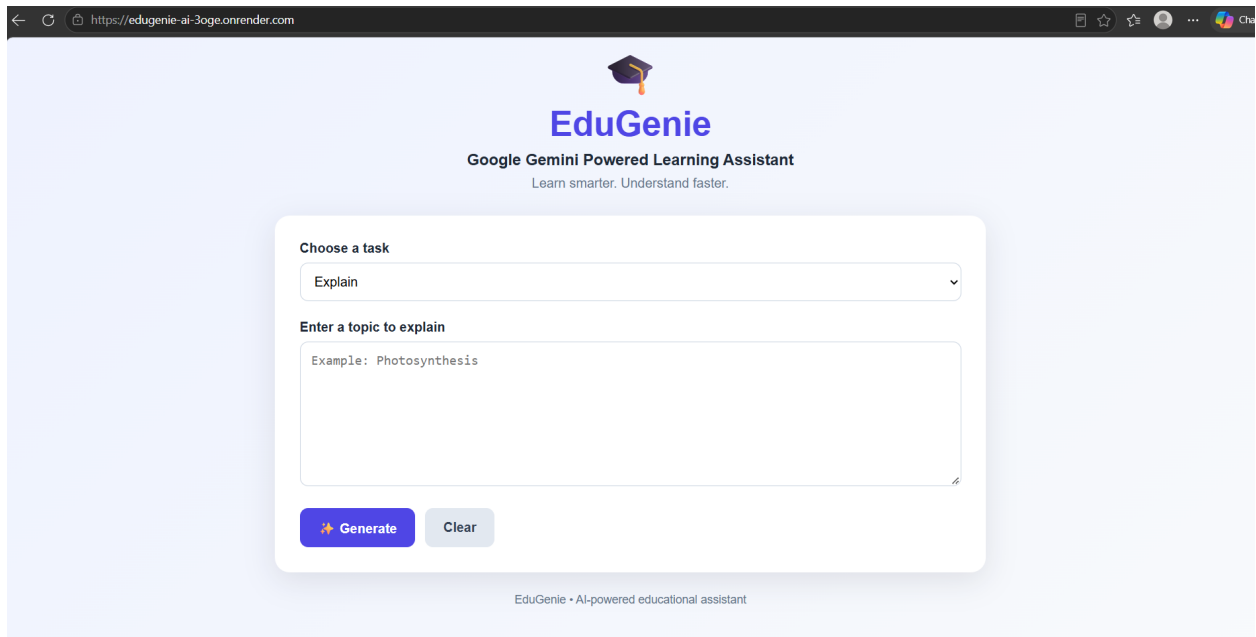


Figure 1. EduGenie landing interface with the five learning tasks available in the selector.

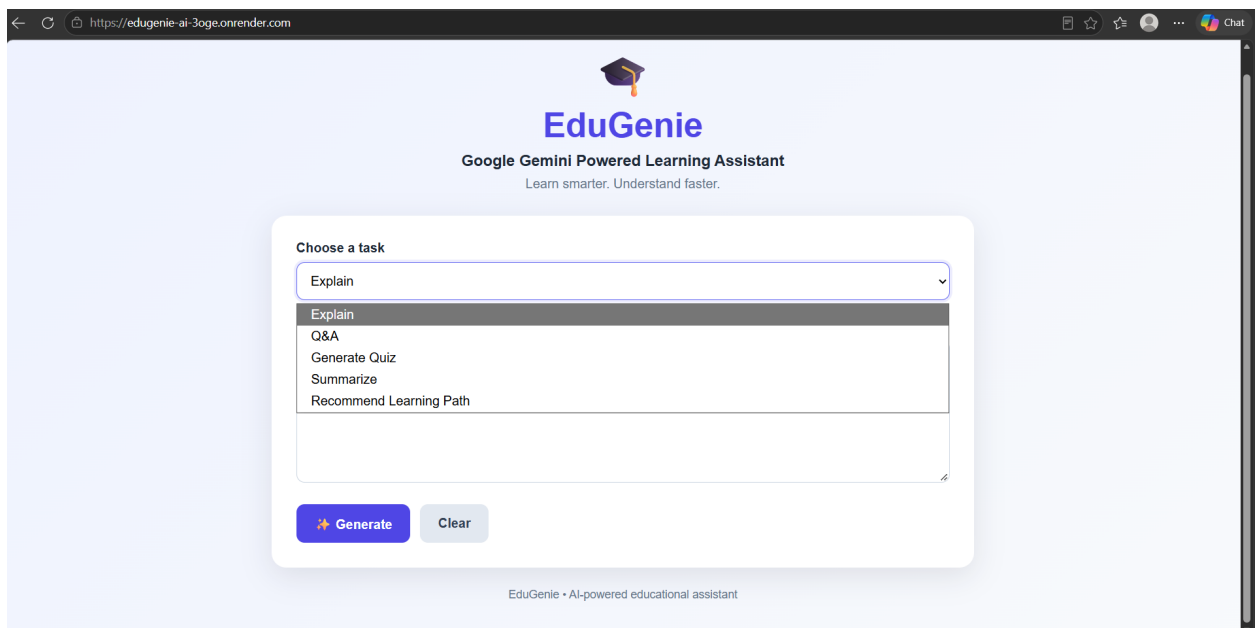


Figure 2. Task selector showing Explain, Q&A, Generate Quiz, Summarize, and Recommend Learning Path.

8.1 Interaction Flow

- Select a task from the dropdown.
- Enter the relevant question, topic, or educational text.
- Click Generate.
- A loading indicator appears while the request is processed.
- The frontend renders the returned answer in the Result section.

- Clear resets the input and result state.
- Errors are displayed in a dedicated error area.

9. Feature Demonstration and Evidence

9.1 Q&A;

The Q&A; interface sends the user question to POST /qa. The tested interface produced a concise explanation for the supplied photosynthesis question.

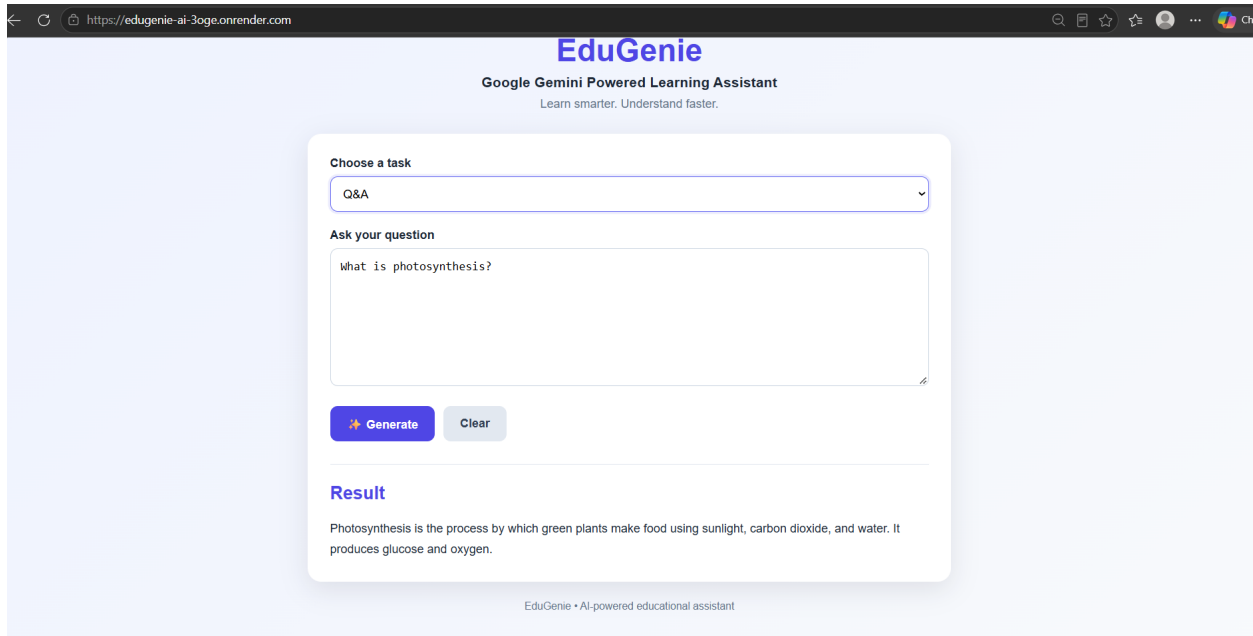


Figure 3. Q&A; feature showing a student-friendly answer.

9.2 Explain

The Explain feature sends a topic to POST /explain. The prompt asks for a structured beginner-oriented explanation.

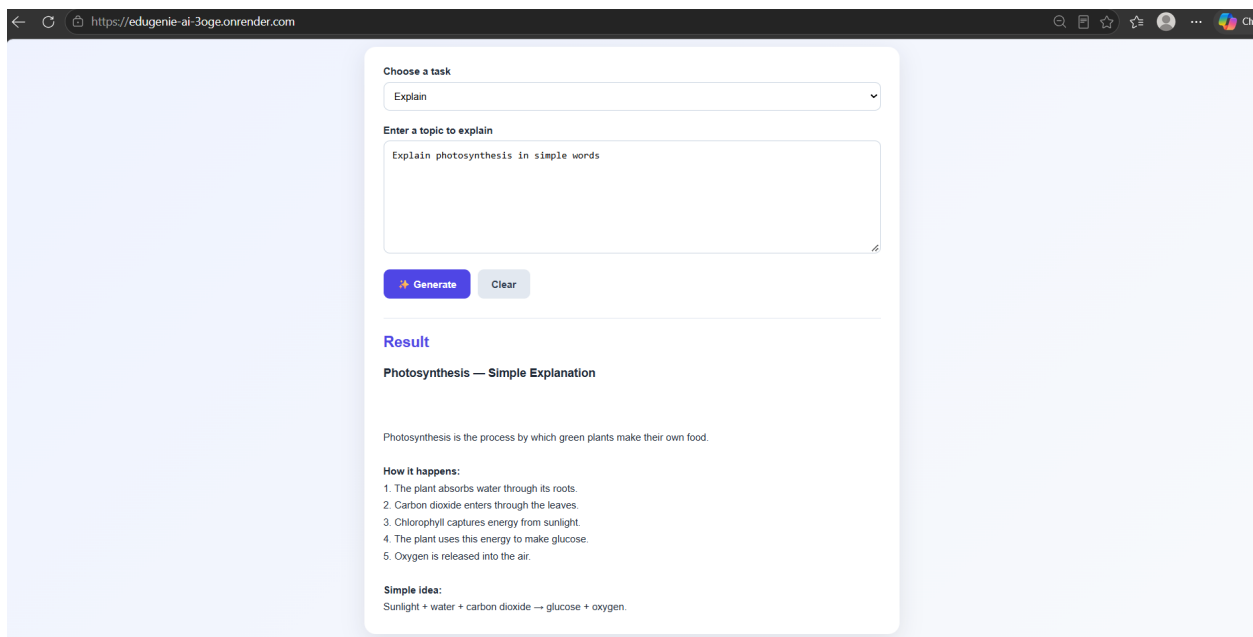


Figure 4. Explain feature showing a structured photosynthesis explanation.

9.3 Generate Quiz

The quiz interface renders the structured response as question cards, options, correct answers, and explanations. The recorded demonstration includes the generated three-question set.

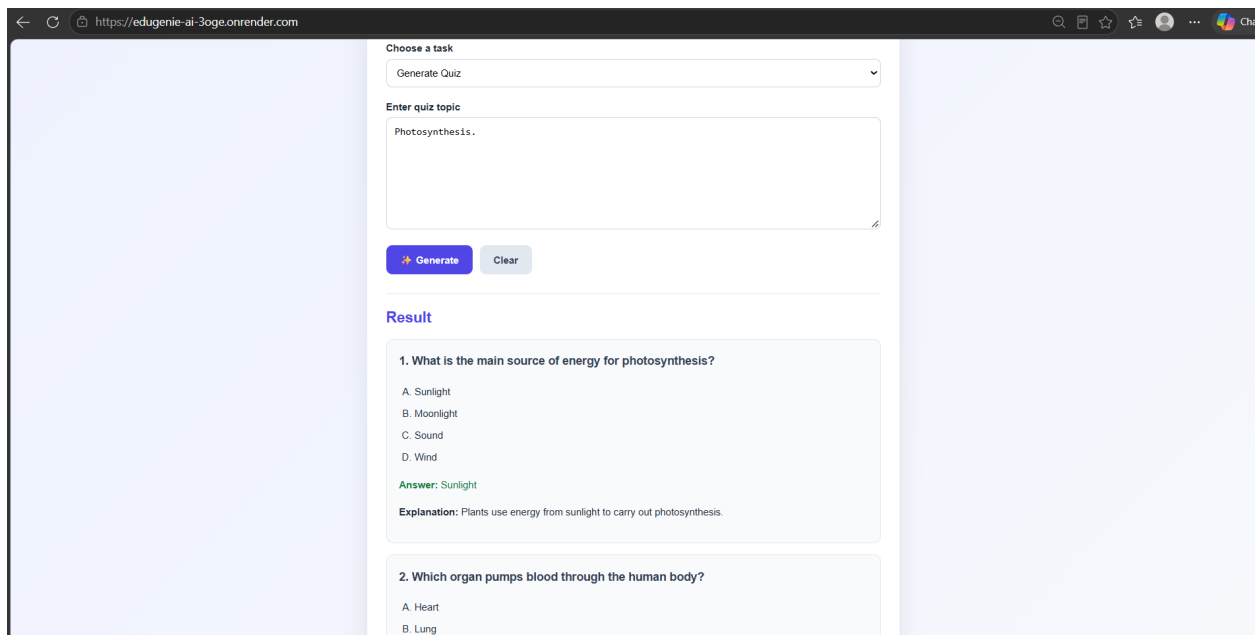


Figure 5. Quiz generation — first portion of the generated MCQs.

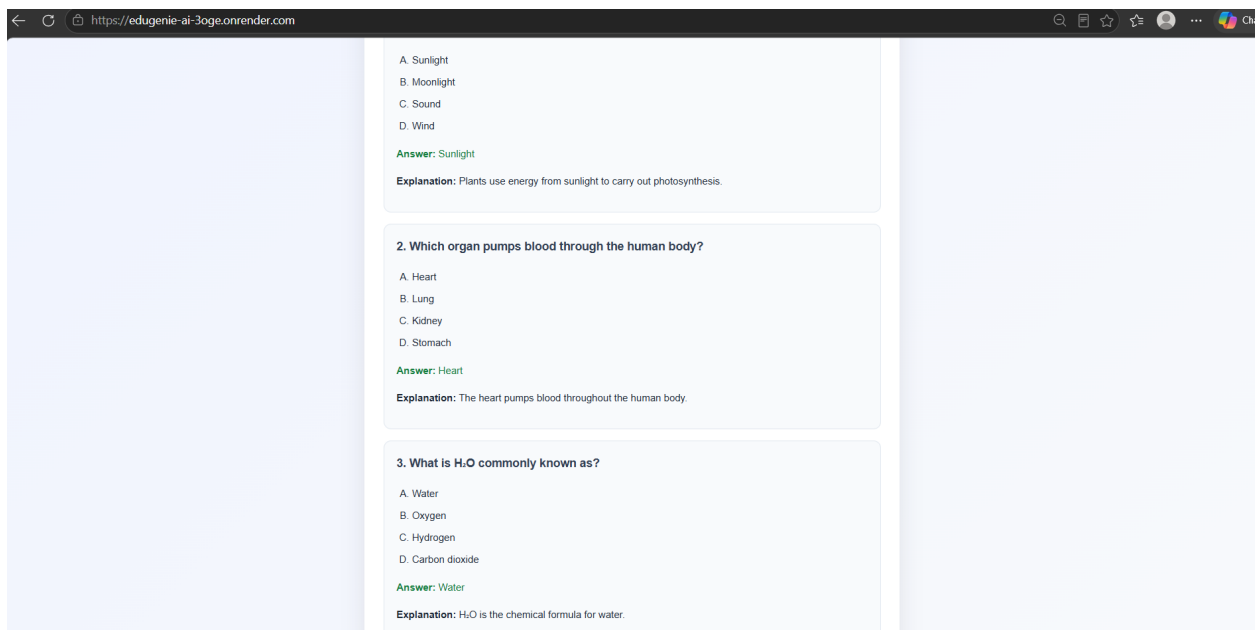


Figure 6. Quiz generation — remaining questions and explanations.

9.4 Summarize

The Summary feature accepts educational text and returns a concise result. The current implementation also has a deterministic fallback that extracts up to four sentences when a temporary AI service condition occurs.

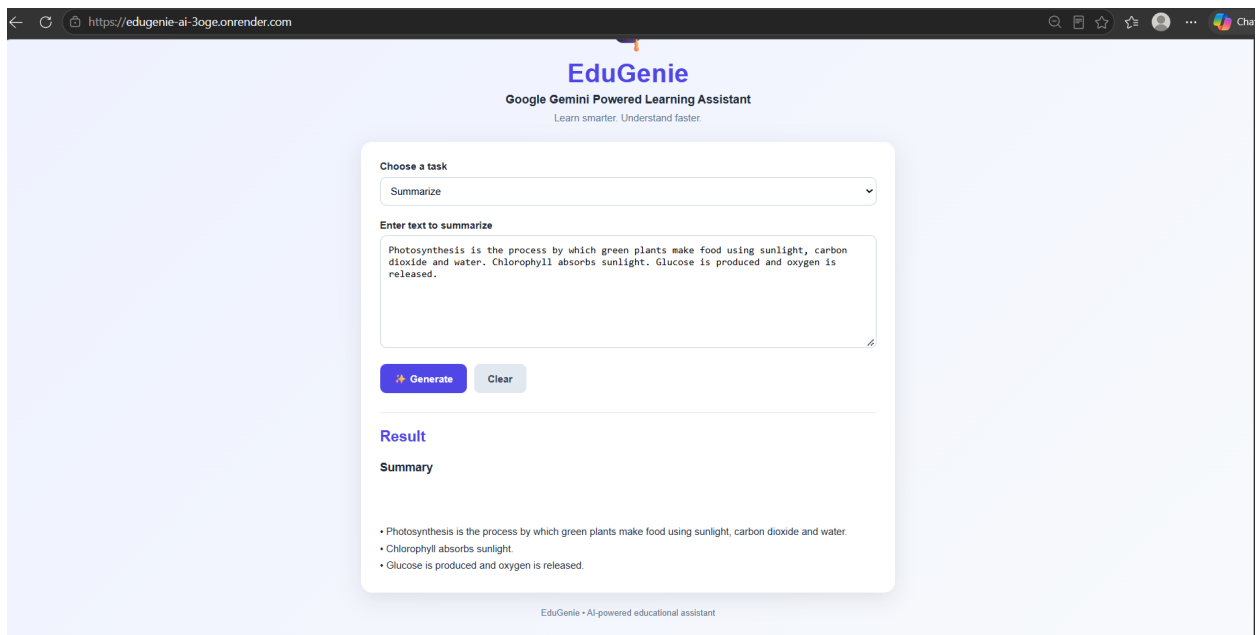


Figure 7. Summary feature showing condensed educational content.

9.5 Recommend Learning Path

The learning-path feature accepts a topic and returns a staged plan. The Gemini prompt requests progression from fundamentals to advanced concepts together with practice and revision guidance.

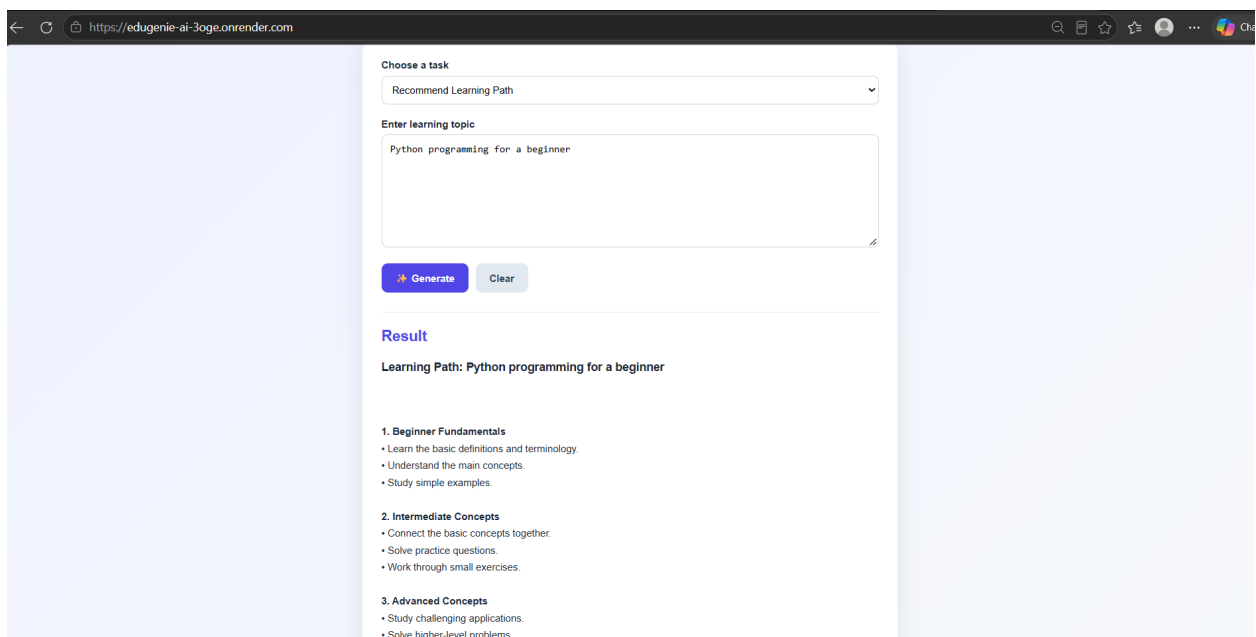


Figure 8. Learning Path feature showing a staged beginner-to-revision plan.

10. Testing and Validation

The application was tested through the live web interface after local development and deployment. The five primary functions were exercised with representative educational inputs. The health endpoint and OpenAPI documentation are also available through FastAPI.

Test Area	Input / Action	Expected Result	Observed Result
Home page	Open /	EduGenie interface loads	Passed
Health	GET /health	JSON status response	Passed
Q&A	Photosynthesis question	Educational answer	Passed
Explain	Explain photosynthesis	Structured explanation	Passed
Quiz	Photosynthesis topic	3 MCQs × 4 options	Passed
Summary	Educational paragraph	Concise summary	Passed
Learning path	Python programming for a beginner	Structured learning plan	Passed
Validation	Empty required input	Validation/error response	Implemented

10.1 Validation and Error Handling

- Empty topic/question/text inputs are rejected at the feature layer.
- Maximum input length is configurable through MAX_INPUT_CHARS.
- Quiz responses are validated for count, option count, and correct-answer membership.
- Temporary Gemini service errors are recognized and routed to fallback behavior.
- The frontend displays a loading state and handles unsuccessful responses.

11. Security and Configuration

The project uses environment variables for sensitive configuration. GEMINI_API_KEY is read through python-dotenv and is not hard-coded into the application source. The .gitignore file excludes .env, virtual-environment files, Python cache files, and pytest cache directories.

- Never commit the real Gemini API key to GitHub.
- Keep the .env file local or configure the secret through the deployment platform's environment-variable settings.
- Do not place API keys in screenshots, documentation, or client-side JavaScript.
- Rotate the key if it is accidentally exposed.

12. Local Installation and Execution

The project can be run using a standard Python virtual environment. The reference development environment used Python 3.14.7 successfully with the supplied dependencies.

- Open the EduGenie project folder in VS Code.
- Create and activate a virtual environment: `python -m venv .venv` and `.venv\Scripts\activate` on Windows.
- Install dependencies with: `pip install -r requirements.txt`
- Create a local .env file containing GEMINI_API_KEY and, if required, GEMINI_MODEL.
- Start the server with: `uvicorn main:app --reload`
- Open `http://127.0.0.1:8000` in a browser.
- Open `http://127.0.0.1:8000/docs` to inspect the FastAPI OpenAPI interface.

13. Deployment and Version Control

The project is maintained in GitHub and deployed as a live web application on Render. The documented live deployment URL is:

<https://edugenie-ai-3oge.onrender.com>

Repository: **<https://github.com/srirammoorthy434/Edugenie-AI>**

The deployment uses the same application architecture as the local project. Environment variables are configured separately from source code so that credentials do not need to be committed to the repository.

14. Limitations

- The current application is a lightweight learning assistant and does not maintain student accounts or persistent learning history.
- The quiz generator currently returns exactly three questions per request.
- There is no database layer in the current implementation.
- The application does not upload or parse PDF/Word study documents.
- Learning-path personalization is prompt-based and does not yet use historical student performance.
- The fallback responses are resilience mechanisms and are not equivalent to live Gemini generation for arbitrary topics.

15. Future Enhancements

- Multilingual explanations and responses.
- Student accounts, progress tracking, and saved learning paths.
- Adaptive quizzes based on previous performance.
- Difficulty controls and configurable quiz length.
- Document upload and retrieval-augmented question answering.
- Learning analytics dashboard with topic mastery and revision history.
- Voice input and text-to-speech support.
- More robust automated test coverage and CI/CD checks.
- Role-based access and stronger production security controls.

16. Conclusion

EduGenie demonstrates a practical Generative AI learning-assistant architecture using FastAPI, Google Gemini, and a lightweight web frontend. The implementation is modular, easy to run, and structured so that each educational capability can evolve independently. The completed workflow covers development, API integration, frontend interaction, validation, resilience handling, testing, GitHub version control, and live deployment.

The project meets the core objectives described in the supplied reference documentation while providing a more complete technical record suitable for project submission, demonstration, and future maintenance.

17. Demo and Presentation Checklist

- Open the live EduGenie URL.

- Show the five available learning functions.
- Demonstrate Q&A; with an academic question.
- Demonstrate Explain with a concept such as photosynthesis.
- Generate a quiz and show all three questions.
- Paste educational text and demonstrate Summary.
- Enter a topic and demonstrate the Learning Path.
- Show the GitHub repository when required for review.
- Keep API credentials hidden throughout the demonstration.

Appendix A — Reference Implementation Notes

The current source package uses the following primary request models: `QAResponse(question)`, `TopicRequest(topic)`, and `TextRequest(text)`. The API returns JSON objects containing success plus the relevant result field for successful calls, or an error field for handled failures.

The optional local explanation model is disabled by default. When enabled, `explanation_module.py` attempts to load MBZUAI/LaMini-Flan-T5-783M through Transformers before falling back to the shared Gemini/fallback path.

The supplied source archive was reviewed for this documentation. A cache artifact under `tests/__pycache__` is not considered part of the application design and is intentionally excluded from the documented source structure.